

FEPGuesser: Feedback-Information Enhanced Password Guesser Based on Natural Language Pre-Trained Model and VAE

Qiuyan Qian[✉] and Jianping Zeng[✉]

Abstract—Creating passwords involves a blend of natural language and password-specific knowledge. Merging these feature to obtain better representations and thus enhancing password cracking efficiency, have consistently remained one of the core challenges in the field of password guessing. In this paper, we put forward the Feedback-information Enhanced Password Guesser (FEPGuesser). We demonstrate, for the first time, how Parameter Efficient Fine Tuning can integrate password knowledge, natural language understanding and bidirectional attention mechanisms to well capture semantic in password sets. Additionally, we propose the novel structure of PassExBertVAE which integrates pre-trained model with Variational AutoEncoder (VAE) architecture for password guessing. We devise the algorithm which can make full use of the inherent properties of the password latent space generated by PassExBertVAE. This algorithm simulates real-world attack scenarios by leveraging attack feedback information to enhance cracking effectiveness. Experiments show that FEPGuesser overall achieves better results than PCFG, FLA, OMEN, PassGAN, PassFlow, DPG and VAEPass. Especially, on the most complex 000webhost dataset, FEPGuesser surpasses the latest PCFG model by 8.75 percentage points and exceeds the DPG model based on representation learning by 34.11 percentage points. Furthermore, cross-site attack experiments show that FEPGuesser is more target-adaptive than PCFG and other deep learning models.

Index Terms—Password guessing attack, parameter efficient fine-tuning, variational autoencoder.

I. INTRODUCTION

PASSWORD guessing is continuously necessary in many circumstances. For instance, in computer forensics, when encountering suspicious files locked by passwords, cracking through password guessing is crucial for working efficiency. Furthermore, password cracking approach also plays an important role in recovering hash encrypted password set. Therefore, improving password cracking efficiency has always been received great attention [1], [2], [3], [4].

Password guessing attacks pose significant threats in various real-world contexts. Some survey papers have already provided

a systematic summary of this [5], [6]. These attacks can be broadly categorized into several scenarios: Trawling attacks aim to crack as many passwords as possible using common or leaked passwords, targeting a wide range of accounts without focusing on specific victims [2], [3], [7], [8]. In targeted guessing, attackers use personal data, like names or social media info, to make educated guesses [9]. Password guessing attacks can be online (interacting with servers) or offline (using leaked password files). Hybrid Password Guessing combines models to improve success rates [10]. Conditional Password Guessing leverages common patterns to fill in the unknown parts of partially known passwords (e.g., completing “d**102” by guessing the missing characters represented by “**”) [4], [11], [12]. Dynamic Password Guessing adjusts strategies based on previous attempts to increase efficiency [4], [11]. Cross-site password guessing refers to the scenario where attackers use credentials from one site to guess passwords on another, exploiting similarities across datasets and thereby targeting all users [13]. In contrast, credential tweaking [14] focuses on targeted attacks against individual users by modeling personalized password modification strategies.

As far as current research is concerned, there are two mainstream approaches to crack passwords. One is based on classic statistical models, such as PCFG, Markov. PCFG model can incorporate many features, such as letter, digital, special character, leet, keyboard sequence, and so on, which enables it to have high success rate. However, these features should be manually defined before model training. Even worse, as password features continue to be mined, the difficulty of discovering new features is increasing. In order to break through the limitations, researchers turn to another guessing method, that is, deep learning.

Deep learning model adopts an end-to-end modeling approach, allowing the model to automatically extract password features through a multi-layer neural network structure. With this method, deep learning is expected to achieve better guessing performance than PCFG. However, this is not the case in reality. The main reason may lie in the fundamental password latent feature space and the corresponding sampling strategy. Early models like FLA [15], PassGAN [3] respectively utilize recurrent neural networks, generative adversarial networks to train on password datasets. The recent work of DPG [4] and PassBERT [12] employs representation learning and BERT’s [16] structure, respectively. The common

Received 8 December 2024; revised 18 March 2025, 14 May 2025, and 12 July 2025; accepted 10 August 2025. Date of publication 18 August 2025; date of current version 22 August 2025. The associate editor coordinating the review of this article and approving it for publication was Prof. Ding Wang. (Corresponding author: Jianping Zeng.)

The authors are with the School of Computer Science, Fudan University, Shanghai 200433, China, and also with the Engineering Research Center of Cyber Security Auditing and Monitoring, Ministry of Education, Shanghai 200433, China (e-mail: qyqian22@m.fudan.edu.cn; zjp@fudan.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3599977

characteristics of these models are the use of character-level modeling methods. However, passwords contain various meaningful components, such as words, sub-words, and abbreviations. Obviously, in the process of model training, entirely learning from characters can lead to many insignificant password features. In addition, the dimensions of the latent space inferred by deep learning are usually very high, therefore, the sampling process is difficult to perceive the distribution of features in each dimension and sub-space. As a result, there exists significant differences between the sampling results and actual password habits.

Since both natural language words and non-natural language components appear in password datasets (this point is demonstrated in Appendix), it is possible to learn the password representation with the help of the rich semantics of natural language. Inspired by research on exBERT [17] within parameter-efficient fine-tuning (PEFT) techniques, we propose that, beyond incorporating a domain-specific vocabulary, domain adaptation parameters should be integrated during fine-tuning to further enhance password representations with natural language cues.

In the paper, we introduce the novel password guess framework FEPGuesser which is able to fully take this advantage. In the FEPGuesser, passwords are first divided into components of words and password-specific components. Then, they are respectively input to natural language pre-trained model and password pre-training layers to obtain their representation, resulting in a final password representation enriched by natural language. The password representation is further transformed into a latent space by means of dimensional reduction in a VAE structure model. Due to the usage of VAE architecture, we can get the capability to control the representation deviation for corresponding password inputs. Subsequently, the obtained new representation can be utilized to generate semantically similar new passwords for attack purposes based on the decoder of VAE. Most important of all, FEPGuesser provides a flexible similar password generation mechanism for a given password. Thus, we can generate more similar, diverse and target-adaptive passwords with the help of attack feedback.

The contributions are summarized as follows:

- 1) We pioneer the concept of natural language-enhanced password representations for password guessing. First, we introduce the Password Segment Vocabulary Generator (PSVG), which enriches traditional password vocabularies with natural language segments, boosting the cracking success rate by 1.13 percentage points (from 38.97% to 40.10%) under 10^7 guesses. Next, we design a natural language-enhanced BERT architecture, adding a password-specific pre-training layer to the standard language model. This yields an 11.12 percentage point improvement in cracking success rate (from 28.98% to 40.10%). Finally, replacing the encoder in PassExBertVAE with PassBert further increases the cracking success rate by 13.12 percentage points (from 26.98% to 40.10%), confirming the effectiveness of our method across both vocabulary and model architecture.

- 2) We pioneer the integration of pre-trained models with VAE structures for password generation, introducing PassExBertVAE as the core of the FEPGuesser framework. This innovative approach leverages latent space to flexibly control password representation and generation. Our exploration of PassExBertVAE's latent space reveals two key properties: locality, where similar passwords cluster together, and semantic drift, demonstrating meaningful password transformations. These findings validate our approach and enhance password guessing strategies through controlled latent space manipulation.
- 3) We introduce a novel target-adaptive sampling algorithm with broad and controllable sampling range, which simulates real-world attack scenarios by utilizing attack feedback information to optimize the success rate. Ablation studies demonstrate significant improvements in cracking efficiency: the dynamic adjustment of sampling strategies based on attack feedback enhances the cracking rate from 39.74% to 40.10% compared to static approaches, and from 39.53% to 40.10% when contrasted with single-round sampling methods. Notably, on the highly complex 000webhost dataset, our algorithm achieves remarkable performance gains, surpassing the state-of-the-art PCFG [7] by 8.75 percentage points and outperforming DPG [4] by 34.11 percentage points. Through comprehensive experimental evaluation across four datasets, we conduct an in-depth analysis from four critical dimensions: password length, segmentation patterns, generated passwords analysis and semantic characteristics. This multi-faceted analysis provides substantial evidence for the algorithm's superior cracking success rate and offers valuable insights into the underlying mechanisms driving its effectiveness.

II. RELATED WORKS

A. Traditional Password Guessing Models

Password guessing attacks were first proposed as early as 1979 [18], during which brute force and dictionary attacks were already prevalent. The popular cracking tools of John the Ripper [19] and Hashcat [20] both have the two attack functions.

Non-deep learning data-driven password guessing models primarily fall into two categories: the PCFG [1] series which focuses on modeling passwords by explicitly decomposing them into multiple LSD components, and the Markov [2] series which primarily models the probability of the next character using Markov chains.

From the perspective of password modeling, a typical representative is LSD structure [1] which divides passwords into three components: letters, special characters, and digital strings. Veras et al. [21] extended the LSD structure by incorporating semantic information. Li et al. [22] analyzed leaked dataset of Chinese passwords and enhanced the PCFG cracking capability by incorporating Chinese Pinyin and rules. This enrichment of the LSD structure has contributed to the advancement of password cracking in the Chinese password domain. Houshmand et al. [23] optimized the LSD

segmentation method by incorporating keyboard patterns and multi-word patterns. In addition, personal identifiable information [9] has also been employed for guessing specific victim passwords. Xu et al. [8] introduced the PwdSegment method, which extended the Byte Pair Encoding (BPE) algorithm to enable automatic decomposition of passwords into multiple chunks. However, these methods require explicit human effort to mine discrete features, aiming to achieve a more diverse segmentation outcome through more detailed features, thereby enriching the forms of generated passwords. They rely on prior knowledge provided by humans and struggle to automatically uncover rich implicit features from data.

Next, we introduce the models of Markov series. Castelluccia [24] first employed the concept of n-grams to capture the essence of passwords. Narayanan and Shmatikov [25] introduced standard Markov model to reduce the searching space of passwords. Later, Duermeth et al. [2] extended the idea of Narayanan and Shmatikov and proposed Ordered Markov Enumerator (OMEN) to generate password candidates according to their occurrence probabilities. Some methods combine these two approaches to achieve better attack effectiveness. Xie et al. [26] introduced WordMarkov, which treats passwords as connected multiple words, leading to more accurate identification of semantic information in passwords while enhancing generalization capabilities. However, the Markov model, based on n-grams, has limitations in capturing long-range dependencies and complex patterns, as the n-gram method primarily relies on fixed-length local contexts.

PCFG and n-gram are both classic natural language representation methods. The former establishes corresponding generative grammars based on the context of the language, while the latter describes the conditional probability distribution of sequential symbols. The success of these models, especially PCFG, in password guessing can demonstrate a strong connection between passwords and natural language knowledge. This is also one of the reasons why this paper introduces natural language embedding into password representation learning.

B. Deep Learning-Based Password Guessing Models

Deep learning is gaining ground in password guessing attacks in recent years. Melicher et al. [15] were the first to introduce the application of Long Short-Term Memory (LSTM) networks in the domain of password guessing, presenting the FLA model. This pioneering methodology incorporates a lightweight architecture, yet it achieves a significant level of accuracy. PassGAN [3] represents a groundbreaking study that employs Generative Adversarial Networks (GANs) in password guessing. The authors train the discriminator using real samples, which allows the generator to iteratively generate distributions that closely resemble authentic password patterns. Pasquini et al. [4] conducted a study on the application of representation learning in password guessing attacks and proposed Wasserstein autoencoder (WAE) method.

PassFlow [27], a novel password guessing approach based on generative flows, was proposed by Pagnotta et al. in 2022. It offers precise latent variable inference, enabling operations such as exploring specific subspaces and interpolation within

the latent space. However, the aforementioned models still solely adopt character-level tokenization, entirely relying on the models to capture the complex patterns between passwords, which poses challenges for representation. VAEPass [28] augments the vocabulary with common character combinations, resulting in an improvement in the efficiency of password guessing. However, this method still has its shortcomings. The aforementioned approaches is trained solely on password datasets, making it difficult for the model to comprehend certain natural language expression patterns that exist between passwords. For example, “520” often signifies “love” in Chinese contexts, which is a common pattern in passwords. Yet, training only on password data makes it challenging to associate “520” with “love”. The CNN encoder used in VAEPass focuses on local character-level patterns through fixed-size convolutional filters, which may limit its ability to capture long-range structural dependencies in passwords.

The idea of pre-training has been utilized in the context of password guessing. Li et al. [29] introduced an architecture comprising five LSTM layers and an output layer. They employed knowledge distillation, utilizing BERT as a tutor, to guide the LSTM network in the process of password generation. The authors demonstrated that this tutoring process by BERT resulted in a substantial improvement in the model’s performance. Xu et al. [12] introduced PassBert, a pre-trained password model learned from password dataset and not equipped with knowledge of natural language. They further fine-tuned the model on three downstream tasks: conditional password guessing (CPG), targeted password guessing (TPG), and adaptive rule-based password guessing (ARPG). The experimental results demonstrate that the fine-tuned PassBert model outperforms existing state-of-the-art models in all three tasks. Although this article employs a natural language model structure, the tokenization remains at the character level, and the model is only pre-trained on a large-scale password dataset. As a result, it still struggles to capture the natural language patterns present in passwords.

C. Parameter Efficient Fine Tuning

With the advancement of computing power, transformer-based pre-trained models, such as BERT [16], RoBERTa [30], ALBERT [31], and SpanBERT [32], have shown remarkable performance in various NLP tasks. However, these models, trained on vast amounts of data, possess a significantly larger number of parameters compared to standard deep models. Fine-tuning large models for different tasks can be expensive in terms of storage and training resources.

To address this challenge, Parameter Efficient Fine-tuning (PEFT) has been proposed. PEFT aims to achieve fine-tuning effects on specific tasks by training only a small subset of parameters. These parameters could be newly added or a subset of the existing model parameters. Lialin et al. [33] have categorized PEFT into three main types: Addition-based, Selection-based, and Reparametrization-based. The primary concept behind Addition-based methods involves the incorporation of extra parameters or layers, with a focus on training exclusively these newly introduced components. The Adapter method and Soft Prompts are two primary techniques within

TABLE I
DESCRIPTION OF NEWLY INTRODUCED TERMS

Symbol	Description
PSVG	Password Segment Vocabulary Generator, a hybrid vocabulary generation algorithm that combines a natural language vocabulary with high-frequency password patterns. Details can be found in Section III-B.
PassExBert	Encoder freezes the pre-trained parameters of BERT, integrates password-specific training parameters into each BERT layer, and dynamically fuses both representations via a gating mechanism. Detailed in Section III-C.
PassExBertVAE	A model that uses PassExBert as the encoder and VAE as the overall framework. Details can be found in Section III-D.
ADEA	Adaptive Density Exploration Algorithm, an algorithm designed to explore the latent space semantic distribution of passwords in PassExBertVAE. Detailed in Section III-E.
ABDPG	Attacking-information Based Dynamic Password Guessing Algorithm, an attack sampling algorithm proposed in this paper. Detailed in Section III-F.
Pivot Password	The initial point used to determine the starting encoding position for sampling in the latent space of PassExBertVAE for the ABDPG algorithm.

the category of Addition-based methods. Our work mainly focuses on the Adapter method approach.

The adapter method was initially applied in the field of Computer Vision [34] and later introduced to the domain of Natural Language Processing by Houlsby et al. [35]. In contrast to full-parameter fine-tuning, the adapter tuning method freezes all parameters of the original model and only trains the additional task-specific adapter modules. This approach reduces the number of parameters that need to be updated during the fine-tuning process. AdapterFusion [36] builds upon Adapter Fine-tuning by integrating multiple adapter-specific information, thereby facilitating the collaborative sharing of multi-task information. Bapna et al. [37] employed adapters to address domain adaptation and large-scale multilingual neural machine translation. In the biomedical domain, exBERT [17] incorporates a domain-specific extension vocabulary and employs a small extension module to facilitate the adaptation of an augmenting embedding for the new domain, within the context of the original BERT's embedding using a general vocabulary. As it integrates both natural language knowledge and domain knowledge while incorporating a domain-specific vocabulary, it outperforms direct full-parameter fine-tuning with only a slight increase in parameter size. Since this aligns with our needs, we draw on exBERT's research methods for our study.

III. APPROACH

In this section, we begin by offering an overview of the FEPGuesser framework. Subsequently, we provide a detailed introduction to each component within the FEPGuesser. Table I presents the main newly proposed terms in this paper.

A. Overview

FEPGuesser, as depicted in Figure 1, encompasses several components for password generation: PSVG, PassExBert, PassExBertVAE, and ABDPG Sampler.

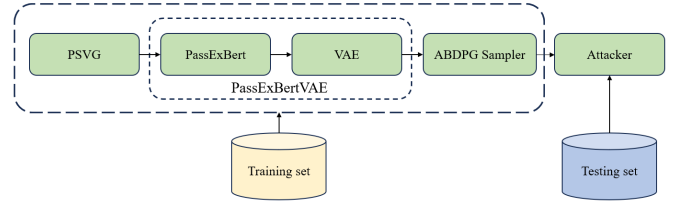


Fig. 1. Framework of FEPGuesser.

- 1) PSVG merges the natural language vocabulary from the BERT pre-trained model with high-frequency patterns found in passwords, forming a hybrid vocabulary. PwdSegment [8] is proposed to divide passwords into several chunks, where a chunk is a sequence of related characters that frequently appear together. Therefore, we adopt PwdSegment as the foundation for PSVG.
- 2) PassExBert first integrates natural language and password representation by freezing the parameters of the original BERT model, introducing additional learnable layers. Then, a gating mechanism is employed to seamlessly integrate the original BERT layers with the newly learned layers, resulting in password representation enhanced through the incorporation of natural language.
- 3) PassExBertVAE employs a VAE approach, facilitating the manipulation of latent variables for passwords. PassExBertVAE replaces the encoder of VAE with the fine-tuned PassExBert, so as to guide the generation of the latent space.
- 4) ABDPG Sampler samples around the latent space representation of a pivot password and then decodes it to generate semantically similar passwords. To lay the groundwork for the design of an effective attack algorithm, we first propose the ADEA in Section III-E, aiming to explore the distribution of passwords in the neighborhood of a single password's latent representation. Through this exploration, we discover the semantic drift characteristic of PassExBertVAE, which reveals the

TABLE II
MAIN SYMBOL DESCRIPTION

Symbol	Description
V_n	original natural language vocabulary
T_{pass}	password training set
τ	password segment frequency threshold
V_{mixed}	Vocabulary combined with natural language segments and password segments
T_{nl}	PassExBert natural language module part
$T_{passmodule}$	PassExBert password module part
t_{pass}	each input password in training dataset
t_μ	sampling mean in the decoding phase of PassExBertVAE
t_σ	sampling variance in the decoding phase of PassExBertVAE
z_{pass}	representation of t_{pass} in the latent space of PassExBertVAE
θ_c	covariance of the perturbation of z_{pass} in the latent space during the ABDPG sampling phase

potential for guided manipulation in the latent space. Building on these insights, we then propose ABDPG algorithm, which enhances the diversity of generated passwords while maintaining the pivot password as a base. It effectively guides meaningful shifts within the password latent space by leveraging attack feedback information.

Table II explains the main symbols used in the model introduction.

B. Password Segment Vocabulary Generator

This section explains the Password Segment Vocabulary Generator.

We use the rule extraction component from PCFG to generate new terms, selecting high-frequency segments as vocabulary candidates. The key matching patterns are as follows:

Keyboard Extracts common keyboard patterns (e.g., “1qaz2wsx”) frequently used in passwords.

Year Includes frequently used year patterns found in password creation.

Leetspeak Captures leetspeak variations where letters are replaced with numbers or symbols, a practice often seen in passwords [38].

We first prepare a train set of plain-text passwords T_{pass} , BERT original natural language vocabulary V_n , and password segment frequency threshold τ .

Following the approaches of PwdSegment [8] and PCFG [7], we train a multi-word detector by recording the co-occurrences among different characters, providing a basis for segmenting smaller multi-word elements within the LSD structure.

For a password, in stage 1, we first extract the known patterns, and then utilize the trained multi-word detector to identify the remaining high-frequency letter sequences, digit sequences, and special character sequences. We also conduct frequency statistics on occurrences of segments.

The multi-word detector here draws inspiration from PwdSegment [8] which utilizes the Byte Pair Encoding (BPE) [39] algorithm to segment unpartitioned sections within the

current password. Specifically, the multi-word detector is a tool that identifies and splits compound words into multiple base words by statistically analyzing word frequencies in training data, filtering base words based on a threshold, and recursively attempting to parse compound words during analysis.

In Stage 2, first, we choose those $frequency(seg) \geq \tau$ as candidates for the final mixed vocabulary generation.

Since BERT pre-trained on natural language datasets inherently possesses a vocabulary of natural language and comes with built-in knowledge of natural language, passwords also contain many natural language segments, such as “love,” etc. What we need to do is to augment the existing vocabulary V_n with selected password-specific segments.

For each t_{pass} in train set T_{pass} , we need to segment it based on the principle of forward maximum matching. We prioritize checking whether the current segment exists in original natural language vocabulary V_n . If it doesn’t, we then examine whether it exists in candidates selected in Stage 1. If it appears in the password segment candidates, then it’s added to V_{mixed} .

C. PassExBert

PassExBert integrates natural language and passwords representation using the approach of Parameter Efficient Fine Tuning (PEFT).

Firstly, we need to tokenize the input password using a tokenizer. Here, we utilize the hybrid vocabulary obtained in Section III-B. Figure 2 illustrates the tokenization process. When segmenting the password “loveliu1314”, the natural language vocabulary can only recognize “love”, while “liu” and “1314” are segmented into smaller subwords by the natural language vocabulary, despite the frequent appearance of these two words in passwords. Therefore, incorporating them into the password vocabulary lays a strong foundation for our subsequent password representation.

As depicted in Figure 3, we augment the representation of passwords by combining the frozen original BERT output and trainable additional module. We use gating mechanisms to combining the natural language module T_{nl} and password

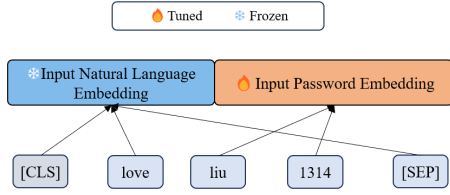


Fig. 2. Application of hybrid vocabulary generated by PSVG. The password “loveliu1314” is divided into three segments: “love”, “liu”, and “1314”. Among these, “love” belongs to the original natural language vocabulary of BERT, while both “liu” and “1314” are part of the additional password-specific vocabulary.

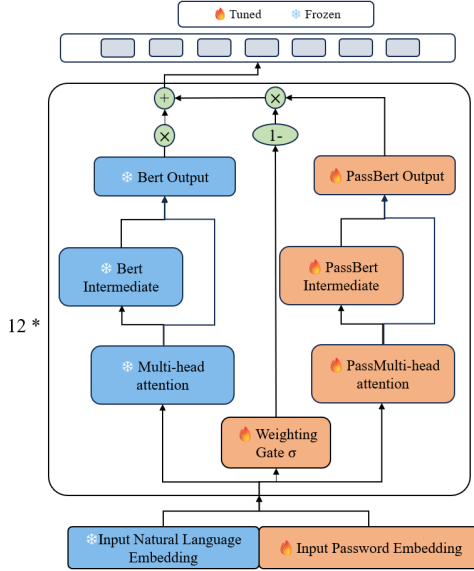


Fig. 3. PassExBert Structure. The Encoder of PassExBert consists of 12 layers. The left part of each layer is a pre-trained BERT model, the parameters of which remain frozen during fine-tuning. This component signifies the representation of input passwords by the natural language model. On the right, a trainable structure mirrors the left section, but each of its components contains only one-third of the parameters compared to its left counterpart. It represents the learned representation from passwords created by human. We employ a gating mechanism to dynamically blend the outcomes from the left and right components, thereby generating passwords representation enhanced with natural language.

module $T_{passmodule}$ as equation 1:

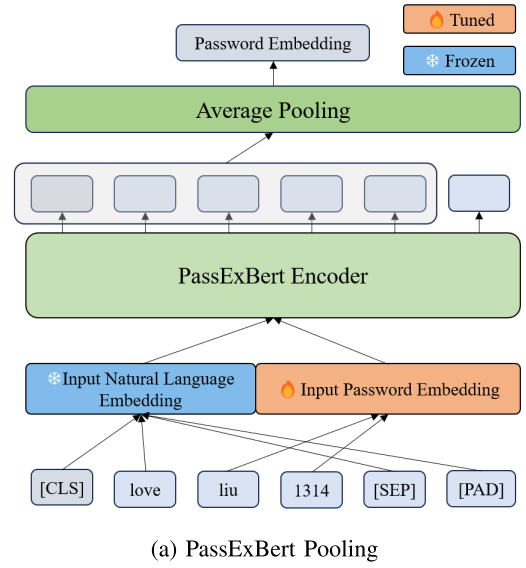
$$h^{l+1} = T_{nl}(h^l) \cdot \sigma(Wh^l) + T_{passmodule}(h_l) \cdot (1 - \sigma(Wh^l)) \quad (1)$$

where h^l is the output of l -th layer. W is the weighting matrix with size 768×1 , which is a fully-connected layer that automatically selects the ratio of output results from the natural language module and the password module based on the input h_l . In order to constrain this ratio between 0 and 1, we opt for the sigmoid function σ .

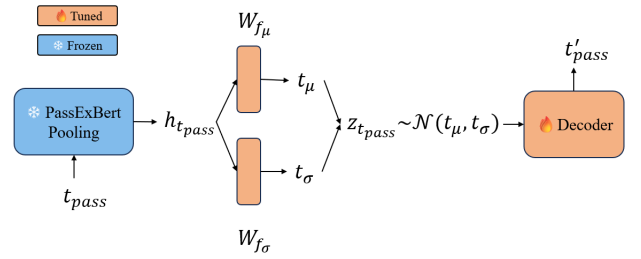
PassExBert utilizes the Masked Language Modeling (MLM) task for fine-tuning on the corresponding password dataset, learning the respective password representations. We describe the specific fine-tuning setting of hyper-parameters in Section IV-B1.

D. PassExBertVAE

After fine-tuning PassExBert, we need to utilize it to construct a latent space, so that we can generate semantically



(a) PassExBert Pooling



(b) PassExBertVAE Structure

Fig. 4. PassExBert pooling and PassExBertVAE structure. (a) Password embedding is average pooling result of PassExBert first and last layer output. (b) In PassExBertVAE, the frozen PassExBert model serves as the Encoder of the VAE. It guides the generation of latent variable $z_{t_{pass}}$, which is then used by the decoder to reconstruct the final password.

similar passwords by controlling latent variables. Therefore, we choose the Variational Autoencoder (VAE) model.

We freeze the parameters of PassExBert fine-tuned in Section III-C. The PassExBert will be employed as the Encoder of PassExBertVAE.

Before training PassExBertVAE, we need to clarify how PassExBert encodes input t_{pass} . Inspired by the bert-flow [40], we choose to encode the input password t_{pass} using the hidden states concatenated by the first and last layers of the PassExBert Layer output. As depicted in Figure 4a, we employ average pooling to aggregate the results from the PassExBert, resulting in the input password embedding $h_{t_{pass}}$.

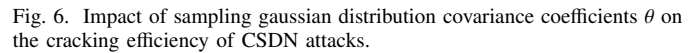
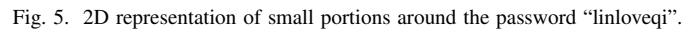
After encoding the input password t_{pass} , just as Figure 4b shows, we use two fully connected layers f_μ and f_σ to map out the mean t_μ and variance t_σ of the input password sample t_{pass} in the latent space. The mapping is respectively denoted as equation 2 and 3. Through the reparameterization technique, we obtain the latent space representation $z_{t_{pass}}$ of the current input password t_{pass} .

$$t_\mu = W_{f_\mu} h_{t_{pass}} \quad (2)$$

$$t_\sigma = W_{f_\sigma} h_{t_{pass}} \quad (3)$$

To ensure the stability of VAE training, we implement the cyclical annealing strategy proposed in paper [41] to mitigate the KL vanishing issue.

Figure 5 illustrates ADEA using t-SNE to visualize average coordinate vectors of newly generated passwords. Starting



To balance efficiency and coverage, we adopt a dynamic θ adjustment strategy—increasing it during attacks to maintain high hit rates while expanding search diversity.

Algorithm 1 Attacking-Information Based Dynamic Password Guessing Algorithm

Input: pivot password set T_{pivot} ,
initial covariance coefficient θ_0 ,
covariance coefficient increasing step size s_θ ,
number of points sampled per each pivot password N_v ,
beam size s_{beam} ,
number of new passwords need to be sampled N_p
Output: newly generated passwords dictionary for attack P_{new} ,

```

1  $\theta_c = \theta_0$ 
2  $S_{att} = \{\}$ 
3  $S_{all} = \{\}$ 
4 for  $t_p \in T_{pivot}$  do
5    $z_t = \text{PassExBertVAE}_{Encoder}(t_p)$ 
6    $P_{new} \leftarrow \text{BeamSearchSampler}(N_v, s_{beam}, \theta_c, z_t)$ 
7    $S_{att} \leftarrow \text{Attack}(P_{new})$ 
8 end
9  $S_{all} = S_{att}$ 
10 while  $\text{len}(P_{new}) \leq N_p$  do
11    $t_{att} = \text{Pop}(S_{att})$ 
12    $z_t = \text{PassExBertVAE}_{Encoder}(t_{att})$ 
13    $P_{new} \leftarrow \text{BeamSearchSampler}(N_v, s_{beam}, \theta_c, z_t)$ 
14    $S_{att} \leftarrow \text{Attack}(P_{new})$ 
15    $S_{all} \leftarrow S_{att}$ 
16   if reach the end of  $S_{att}$  then
17      $S_{att} = S_{all}$ 
18      $\theta_c = \theta_c + s_\theta$ 
19   end
20 end

```

F. Password Sampling Process

In real-world attacks, leaked passwords typically cover only a subset of a website's credentials. Attackers often launch dictionary attacks using these leaks and iteratively refine their strategies based on success. Similarly, PassExBertVAE begins sampling with training set passwords as pivots and uses successful guesses to improve precision in later iterations.

Based on these findings and the phenomenon of password semantic drift mentioned in section III-E, we propose the ABDPG algorithm to increase cracking rate and efficiency. This approach primarily focuses on two optimization directions.

- 1) Utilizing attack feedback information to position the sampling starting point in areas with higher attack value.
- 2) Employing the beam search [42] algorithm to mitigate semantic drift while generating a diverse range of passwords.

The specific implementation details are outlined in Algorithm 1.

In prior research [4], password decoding utilized a greedy strategy, where the highest-probability token is selected at each step to construct the final string. However, we have observed that greedy decoding often overlooks semantically similar passwords. Simply increasing the covariance coefficient θ may mitigate this issue but can also cause semantic drift. To address this, we integrate the beam search [42] algorithm into ABDPG, enhancing the ability to decode semantically related passwords.

Algorithm 1 outlines the integration of beam search into ABDPG, while Figure 7 illustrates the decoding process.

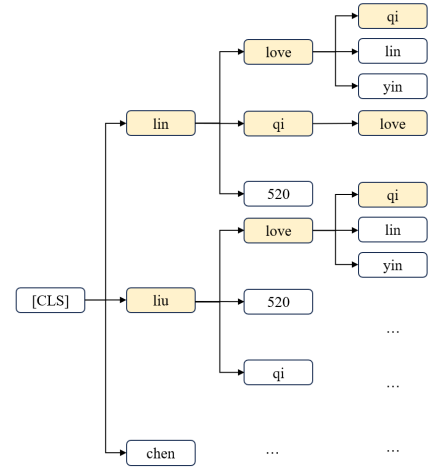


Fig. 7. Beam search procedure: Given the input “liloveqi,” the decoder starts with [CLS] and iteratively selects top-k tokens (e.g., “lin,” “liu,” “chen”) based on Gaussian-sampled encoding. The process continues until [SEP], producing final passwords like “linloveqi,” “linqilove,” and “liuloveqi”.

TABLE III
BASIC INFORMATION ABOUT DATASETS

	Language	Size	Type	Year
CSDN	Chinese	4,410,120	Programming	2011
RenRen	Chinese	3,229,414	Social forum	2011
000webhost	English	15,207,242	Web Hosting	2015
Rockyou	English	32,508,007	Gaming	2009

ABDPG consists of two attack phases: the first-round attack and the dynamic attack.

In the first-round attack, passwords from the training dataset form the pivot set T_{pivot} , from which new passwords are sampled using beam search. After each sampling round, successfully cracked passwords are added to the success stack S_{att} .

In the dynamic attack phase, S_{att} serves as the new pivot set. Beam search is used to generate additional passwords, which are then tested on the target dataset. Successfully cracked passwords are added to S_{att} and collected into S_{all} . When S_{att} is exhausted, S_{all} reinitializes it, and the covariance coefficient θ_c is adjusted for the next round. This iterative process accumulates new cracked passwords unique to each round.

By leveraging attack feedback, this approach accelerates sampling efficiency and better simulates real-world attack scenarios.

IV. EXPERIMENT AND RESULT

A. Dataset

Numerous major password leaks have occurred to date. We conduct experiments using leaked datasets previously studied in related work [4], [12], [38], [43].

For ethical and security reasons, all processing is done offline, and only plaintext passwords are used—excluding any sensitive user data.

Table III presents an overview of the datasets. Originating from different industries, they exhibit diverse password

TABLE IV
VALUES OF τ IN THE PSVG ALGORITHM
FOR DIFFERENT DATASETS

	CSDN	RenRen	000webhost	Rockyou
τ	40	15	200	150

distributions, offering a broad evaluation ground for FEPGuesser. Section IV-F provides a detailed quantitative comparison of these datasets.

We randomly split each dataset into 70% training and 30% testing. Following [44], we do not remove overlapping passwords between train and test sets to maintain realistic password distributions, as common passwords may appear in both. Only passwords containing unsupported characters are removed; no other preprocessing is applied. Unlike models like PassGAN [3], DPG [4], or PassFlow [27] which limit generation smaller than 10 characters, we retain longer passwords, as their inclusion is crucial for comprehensive evaluation. All models are trained on the same data for fair comparison.

B. Experimental Settings

1) *Parameter Settings*: Table IV lists the τ values in PSVG for different datasets, influenced by password count, length distribution, and segmentation. We select τ to keep the number of high-frequency segments close to BERT's vocabulary size.

For PassExBert, we set the MLM mask rate to 0.4 and train for 5 epochs. Following [17], 33.2% of the natural language layer size is used for the password layer, which is fine-tuned while keeping the original layers frozen. PassExBertVAE is trained for 10 epochs.

In ABDPG sampling, training set passwords serve as initial points. Each is encoded, mapped to latent space, and used to sample 2 new vectors from $\mathcal{N}(z_{pass}, \theta I)$ with $\theta_0 = 0.01$ and step $s_\theta = 0.03$. Beam size is set to 5 for decoding.

Each dataset is tested with an order of magnitude 10^7 guesses to maximize cracking success within a smaller number of guesses. Our source code is available on <https://github.com/carolqian31/FEPGuesser>

2) *Baseline Model*: For the baseline models, we select three representative types: one focusing on password structure, PCFG [1], another being the Markov-based model, OMEN [2], and the third focusing on the intrinsic semantics of passwords, deep learning-based models. For deep learning models, we have selected FLA [15], PassGAN [3], PassFlow [27], DPG [4], and VAEPass [28].

We introduce the baseline models and their configurations below. To ensure fair comparison, we set the maximum password length to 34 across all models. Other shared parameters are also adjusted accordingly when necessary.

PCFG: We adopt PCFG V4.3 [7], which extends the original 2009 version [1] with additional segmentation patterns (e.g., year, leets, website, keyboard). These improvements significantly enhance its guessing performance.

OMEN: We use the default n-gram size of 4.

FLA: Following the original paper, we use a 2-layer LSTM with hidden size 1000 and train for 3 epochs. Password lengths

TABLE V
CRACKING RATES OF PCFG, DPG, PASSFLOW, PASSGAN, OMEN, FLA, VAEPASS AND FEPGUSSER ON 000WEBHOST, ROCKYOU, CSDN AND RENREN DATASETS

	000webhost	Rockyou	CSDN	RenRen
Attack Model	Cracking Rates(%)	Cracking Rates(%)	Cracking Rates(%)	Cracking Rates(%)
PCFG	27.59	64.7	40.23	48.95
DPG	2.23	8.19	8.53	37.94
PassFlow	0.25	11.79	21.02	37.71
PassGAN	4.06	22.78	20.65	33.5
OMEN	12.02	39.80	29.59	39.12
FLA	0.27	67.20	14.30	14.07
VAEPass	7.82	31.06	20.67	33.11
FEPGuesser	36.34	63.15	42.08	53.04

are constrained between 1 and 34. Generation uses the model from the final epoch.

PassGAN: This GAN-based model is configured with a max length of 34. Other settings (e.g., 128-dim layers, discriminator update ratio of 10:1) follow the original defaults.

PassFlow: A Flow-based model trained for 10 epochs, with learning rate 0.001 and weight decay 5×10^{-5} .

DPG: Based on [4], DPG uses a GAN and dynamically adjusts its sampling distribution by mixing isotropic Gaussian distributions centered on successfully cracked passwords. In contrast, our method uses each cracked password as a new center with a variable covariance coefficient θ , offering simpler implementation, lower overhead, and more precise exploration around success points. See Figure 6 for analysis.

VAEPass: As the official code is unavailable, we re-implement the *VAEPass_{token}* version based on [28]. Hidden size is set to 512. We train for 30 epochs with learning rate 0.0001. Token frequency threshold is 0.0015, and we supplement the vocabulary with all necessary ASCII characters to ensure correct tokenization.

C. Cracking Results

In Figure 8, we compare the performance of FEPGuesser, the latest PCFG [7], DPG [4], PassGAN [3], PassFlow [27], OMEN [2], FLA [15], VAEPass [28] on the datasets 000webhost, Rockyou, CSDN, and RenRen.

Figure 8a shows the cracking success rate on the 000webhost dataset. At 4×10^7 guesses, the cracking success rate for FEPGuesser is 36.34%, for PCFG it's 27.59%, and for DPG it's 2.23%. At the same number of attack attempts, our method's success rate on the 000webhost dataset increased by 8.75 percentage points compared to PCFG (from 27.59% to 36.34%) and increased by 34.11 percentage points compared to DPG (from 2.23% to 36.34%). Figure 8b, Figure 8c, Figure 8d respectively illustrate the comparative results on the Rockyou, CSDN and RenRen datasets. The specific outcomes are presented in Table V.

It can be observed that the attack effectiveness is highly correlated with the dataset. In Section IV-F, we provide a detailed analysis of the cracking results in relation to the dataset.

D. Ablation Study

1) *Model Structure Ablation Study*: In order to investigate the effectiveness of added components, we conduct

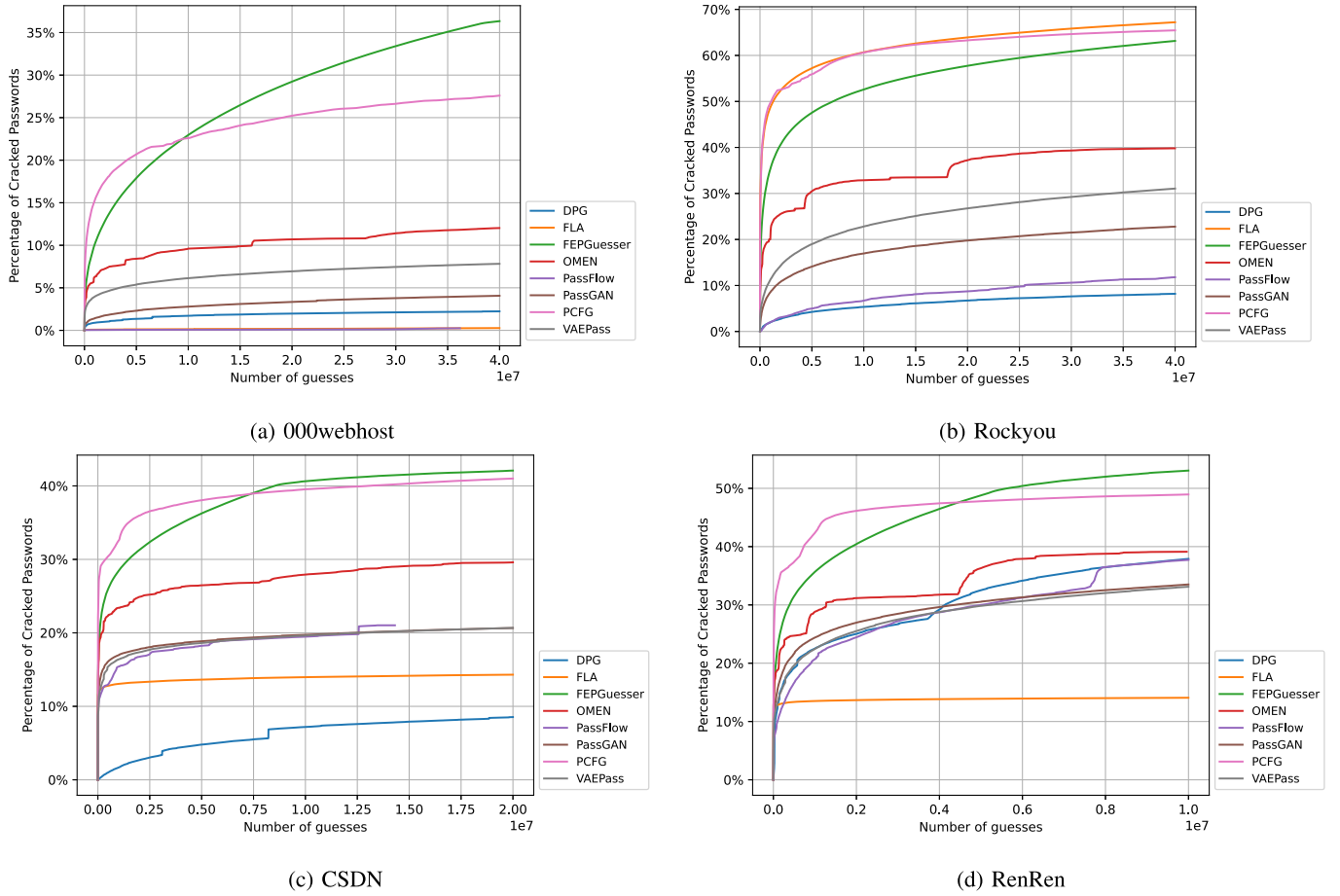


Fig. 8. Cracking results. We compare the cracking success rate of DPG, FLA, FEPGuesser, OMEN, PassFlow, PassGAN, PCFG and VAEPass on the 000webhost, Rockyou, CSDN and Renren datasets. The horizontal axis of the graph represents the number of guesses, while the vertical axis represents the success rate of attacks on the test set.

ablation experiments on the CSDN dataset. We compare our PassExBertVAE with three variations as follows:

- **PassExBertVAE w/o password vocabulary.** PassExBertVAE with PassExBert fine-tuned by removing only the passwords vocabulary. The purpose of this ablation model is to observe whether adding high-frequency password patterns to the vocabulary has a positive impact on the success rate of password guessing.
- **PassExBertVAE w/o password module.** We retain the password vocabulary in PassExBertVAE but remove the additional password layer in the encoder. All original BERT parameters and password vocabulary embeddings are fine-tuned. This setting evaluates whether fully tuning pretrained parameters—without gated fusion—improves password representation and cracking performance.
- **PassBertVAE** In PassExBertVAE, we replace the encoder with PassBert [12] and use a character-level vocabulary for consistency. PassBert is fine-tuned on the same password data as PassExBert, then frozen during decoder training. This ablation model, without any pretraining parameters, is trained from scratch and used to assess the impact of natural language pretraining on password modeling and guessing performance.

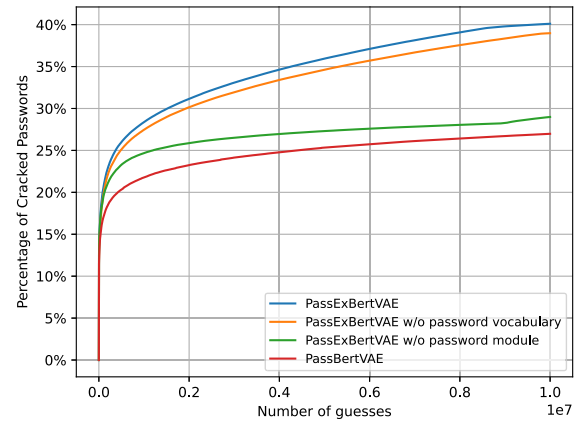


Fig. 9. Results of the ablation experiments on CSDN dataset. Compared the original PassExBertVAE, PassExBertVAE w/o password vocabulary, PassExBertVAE w/o password module.

We all use the ABDPG algorithm to sample 10^7 unique passwords for attacks on CSDN dataset, and the results are shown in Figure 9.

The cracking rate of PassExBertVAE, PassExBertVAE w/o pass vocabulary, PassExBertVAE w/o password module,

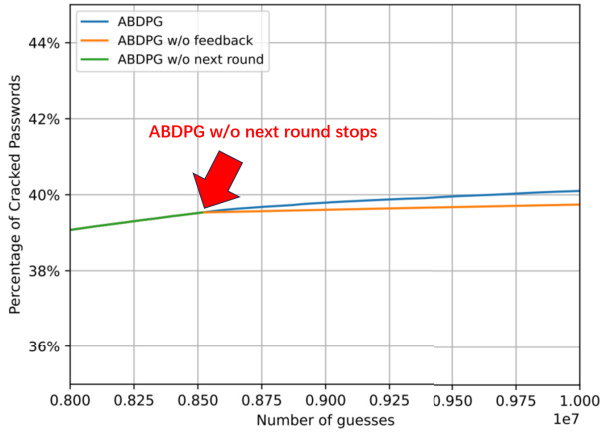


Fig. 10. Ablation experiment results on CSDN dataset: Comparison of ABDPG, ABDPG w/o feedback, ABDPG w/o next round.

PassBertVAE is 40.10%, 38.97%, 28.98% and 26.98% respectively. Thus, adding the password vocabulary resulted in a 1.13 percentage point increase in cracking rates (from 38.97% to 40.10%), while adding and fine-tuning only the additional password layer and keeping original natural language layer frozen, instead of fine-tuning the original natural language layer, lead to a 11.12 percentage point increase (from 28.98% to 40.10%). Compared to PassBertVAE with PassBert as the Encoder, the improvement is 13.12 percentage points (from 26.98% to 40.10%). This experiment demonstrates the effectiveness of adding our module.

2) *Sampling Process Ablation Study*: To validate the effectiveness of our proposed ABDPG algorithm, we conduct additional ablation experiments on the Password Sampling Process. Here, we primarily verify the impact of feedback on the cracking success rate.

We use the PassExBertVAE structure, consistent with the ablation experiments above, and sampled 10^7 unique passwords from the CSDN dataset for attacks.

We compare the original ABDPG algorithm with two ablation algorithms:

- **ABDPG w/o feedback** This variant removes attack feedback from ABDPG, using leaked training passwords as fixed starting points instead of previous successful guesses. Sampling diversity is achieved by increasing the covariance θ_c .
- **ABDPG w/o next round** This algorithm performs a single round of sampling, using leaked passwords from the training set as initial points and a fixed sampling range (θ_c). The number of sampled passwords is limited, unlike the other two algorithms that can generate unlimited passwords.

The cracking success rate for sampling 10^7 passwords using ABDPG, ABDPG without feedback, and ABDPG without the next round are 40.10%, 39.74%, and 39.53%, respectively. Attack diagram is shown in Figure 10. This ablation experiment demonstrates the effectiveness of using successful attack passwords as starting points and expanding the sampling range to generate more diverse passwords.

E. Cross-Site Attack

We also conduct several cross-site attack experiments. Considering the differences in language environments, we fine-tune the models using the 000webhost and Rockyou datasets, both of which are English datasets, and then perform attacks on the other dataset, i.e., fine-tuning on 000webhost and attacking on Rockyou, and vice versa. For each method, we sample 10^7 unique passwords.

To explore the impact of different initial points, we employ two methods for selecting pivot passwords:

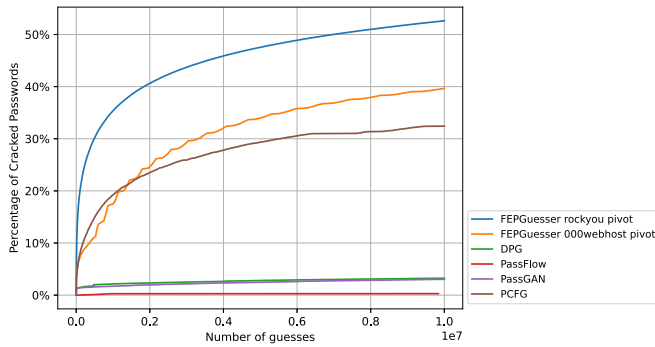
- 1) Selecting the training set of the attack target as the pivot passwords dataset (e.g., training on 000webhost, using the Rockyou test set as the attack data, and the Rockyou training set as the initial point data, with the same train-test split method as described in Section IV-A).
- 2) Selecting the top 10^5 high-frequency passwords from the fine-tuning dataset as the pivot passwords dataset (e.g., training on 000webhost, using the Rockyou test set as the attack data, and the top 10^5 high-frequency password set from 000webhost as the initial point data).

The results are shown in Figure 11.

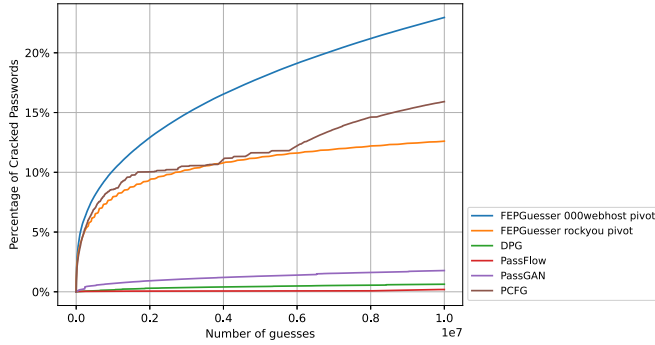
When trained on the complex 000webhost dataset and attacking Rockyou, FEPGuesser consistently outperforms PCFG regardless of initial point selection. However, when trained on Rockyou and attacking 000webhost, using Rockyou passwords as pivots slightly underperforms compared to PCFG. This indicates that complex datasets offer richer latent spaces, enabling broader and more diverse password sampling. In contrast, simpler pivot sets like Rockyou limit sampling diversity, making PCFG's frequency-based approach more effective in such cases.

To further evaluate the effectiveness of FEPGuesser on more datasets, we select the more recent ignis dataset (leaked in 2020) [45] as the target for a cross-dataset attack. Specifically, 70% of the ignis dataset is used as the test set, while the remaining 30% is used to construct the pivot password set for the FEPGuesser-ignis-pivot model. This setup allows us to assess the impact of utilizing initial points that share a distribution more similar to the target test set. The model is trained on the 000webhost dataset and applied to attack the Ignis dataset. The results are presented in Figure 11c. As shown, regardless of whether the pivot passwords are derived from the 000webhost dataset or from a portion of the ignis dataset, our proposed FEPGuesser consistently outperforms DPG, PassFlow, PassGAN, and PCFG in terms of attack effectiveness. This further demonstrates the superior performance and generalization capability of our model on more recent datasets.

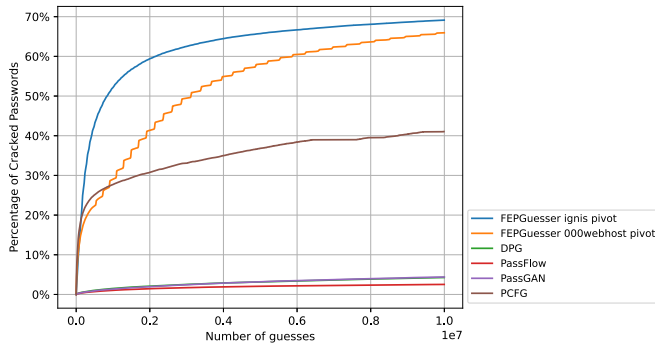
By comparing the attack curves in Figure 11, we find that FEPGuesser performs better with higher-quality pivot passwords that better match the attack data's semantics. This underscores the importance of initial point selection and shows FEPGuesser's potential as a complementary model by using successful attack results from other models to generate more semantically diverse passwords.



(a) Fine-tuning on 000webhost and attacking on Rockyu



(b) Fine-tuning on Rockyu and attacking on 000webhost



(c) Fine-tuning on 000webhost and attacking on ignis

Fig. 11. Cross-site cracking results.

F. Dataset Analysis and Results Interpretation

From Table V, we see that different models exhibit varying cracking success rate on different datasets. This is likely correlated with the inherent characteristics of the datasets themselves. Therefore, we conduct in-depth analysis on the properties of different datasets.

1) *Password Length Analysis*: Figure 12 displays the distribution of password length across the four datasets: 000webhost, Rockyu, CSDN and RenRen.

000webhost passwords tend to be longer, especially in the 15-17 character range, suggesting more complex passwords due to its user base's focus on security. In this dataset, FEPGuesser outperforms other methods.

Figure 13 compares the length distributions of generated and cracked passwords from PCFG and FEPGuesser. Figure 13a compares generated password length from the two models with the original 000webhost password distribution, which

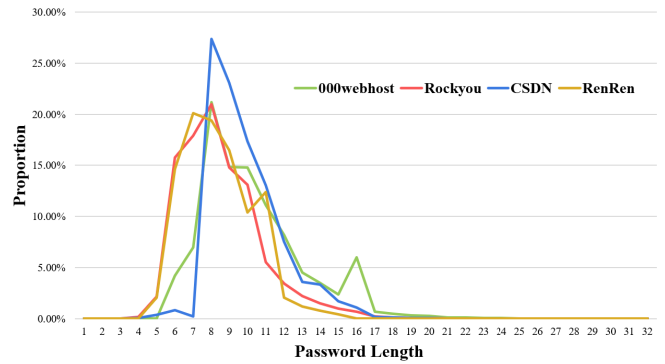


Fig. 12. Distribution of password length in the test set of 000webhost, Rockyu, CSDN and RenRen.

shows a peak in the 15-17 character range. FEPGuesser, using training set initial points, better captures abrupt changes in length distribution due to the similarity between train and test sets. PCFG, focusing on password structure, produces lower probabilities for longer passwords with more segments.

Figure 13b shows FEPGuesser cracks more longer passwords than PCFG, aligning with the 000webhost dataset's characteristics.

In Figures 13d and 13c, although FEPGuesser generates passwords closer to the Rockyu length distribution, PCFG cracks more high-frequency passwords quickly due to its probability-based approach. Longer, rarer passwords in Rockyu are harder to attack, prompting further analysis of password segmentation in the next section.

2) *Password Segmentation Analysis*: The success rate of password guessing models, particularly PCFG, is closely tied to password segmentation, as the probability of generating new passwords depends on the grammar and segment occurrence. Thus, analyzing password segmentation is crucial.

We segment the password dataset using the vocabulary from Section III-B and summarize the details in Table VI, where we calculate the following metrics: average segment length, average number of segments, number of segments from natural language vocabulary, natural language vocabulary utilization, and number of segments from password vocabulary.

We can draw several conclusions:

- The 000webhost dataset has shorter segments and more segments per password, indicating more complexity.
- English datasets show higher utilization of natural language vocabularies than Chinese datasets, benefiting pre-trained models.
- Chinese datasets have longer tokens, gaining more from adding password segments to the pre-trained vocabulary.

As seen in Table V, the complexity of the 000webhost dataset lowers model success rates, especially for PCFG, which depends on structure. Table VI explains why PCFG's performance drops significantly here. PCFG's segment probability is the product of grammar and segment occurrence probabilities. This metric shows that PCFG performs less effectively on 000webhost, where passwords have more segments and shorter lengths.

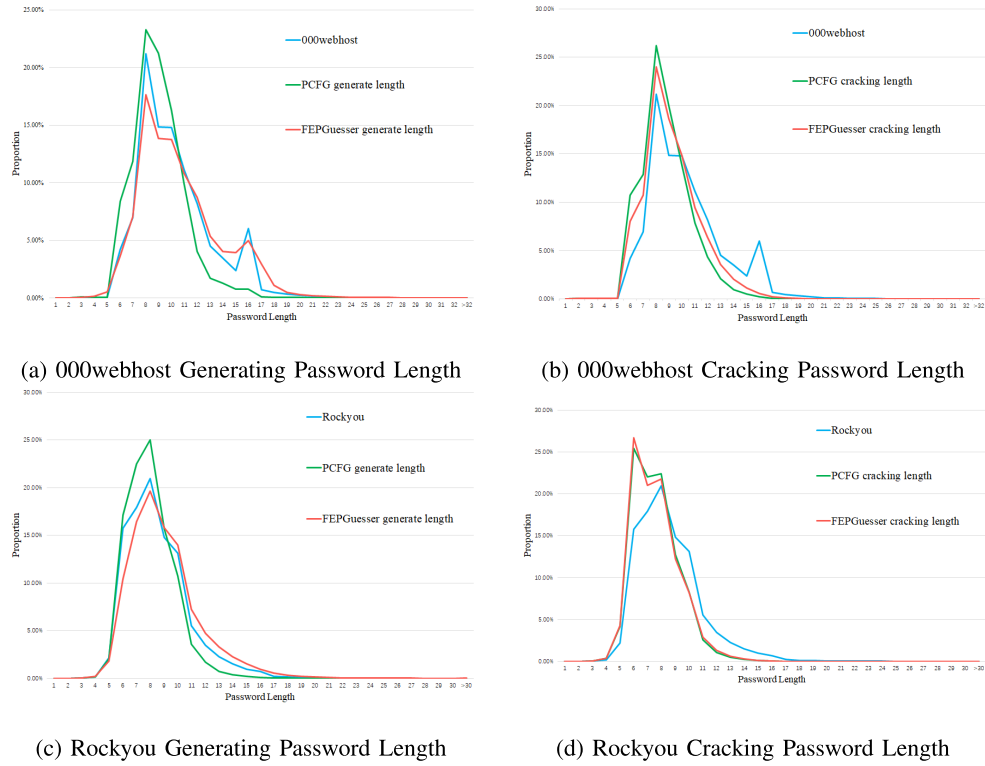


Fig. 13. Comparison of length distributions for passwords generated and cracked by PCFG and FEPGuesser on the 000webhost and RockYou datasets.

TABLE VI
PASSWORD VOCABULARY SEGMENTATION DETAILS AND PCFG PROBABILITY PRODUCT RESULTS

Dataset	average token length	average token num	average NLP token num	NLP vocabulary utilization(%)	average password token num	average PCFG probability(%)
CSDN	3.85	2.97	1.03	34.68	1.94	0.2885
RenRen	3.67	2.45	0.76	31.02	1.69	0.3076
000webhost	2.59	4.47	2.81	62.86	1.65	0.00023
Rockyou	3.61	2.54	1.34	52.76	1.2	0.011

TABLE VII
PASSWORD FREQUENCY COUNTS ANALYSIS

Dataset	Total count	Unique password count	Non-tail password count	Non-tail password percentage(%)	Non-tail password frequency count	Non-tail password frequency percentage(%)
CSDN	3,087,039	2,063,065	176,775	8.56	1,200,749	38.89
RenRen	2,260,591	1,394,992	178,468	12.79	1,044,067	46.18
000webhost	10,645,023	7,758,597	1,025,057	13.21	3,911,483	36.74
Rockyou	22,755,605	10,591,967	1,678,381	15.84	13,842,019	60.83

However, in Table VI, we observe that despite the higher average PCFG probability in two Chinese password sets, CSDN and RenRen, compared to Rockyou, the success rate of PCFG attacks is lower.

To explore this further, we add frequency data in Table VII. “Non-tail password count” refers to passwords appearing more than once, and “Non-tail password frequency count” is the cumulative frequency of these passwords. In the Rockyou dataset, 15.84% of non-tail passwords account for 60.83% of the total, indicating a much higher rate of repetition than in other datasets. This leads to greater success in attacking high-frequency passwords, making probabilistic methods like PCFG and FLA more effective on Rockyou.

3) *Generated Passwords Analysis*: Section IV-F2 analyzes the relationship between PCFG attack effectiveness

TABLE VIII
PASSWORDS SUCCESSFULLY ATTACKED BY FEPGUesser BUT NOT BY PCFG

features	h1gh	g0d	d0g
passwords	h1ghway	G0disg00d	wh1ted0g
	h1ghbury	spellg0d	d0gf00d
	Trav3l1ngh1gh	G0dwithus	l0std0gs
	H1ghland	flameg0d	buld0ga
	H1ghland5	G0d1sl0ve	tappd0gg
	h1ght1m3s	g0dbl3ss	d0ghouse

and datasets. We also examine why FEPGuesser outperforms PCFG and DPG on 000webhost, identifying key factors that enhance cracking rates on complex datasets.

TABLE IX
PASSWORD DATASET FID SCORE

Dataset	All passwords FID	Tail passwords FID
CSDN	27.98	20.77
RenRen	61.44	47.95
000webhost	118.93	89.89
Rockyou	8.42	4.97

Table VIII shows passwords successfully attacked by FEPGuesser but failed by PCFG, featuring elements like “h1gh”, “g0d”, and “d0g”. FEPGuesser outperforms PCFG, especially on leetspeak passwords, which PCFG struggles with due to manual setup and lower occurrence probability, leading to longer segments and shorter average segment lengths. FEPGuesser automatically discovers relevant features from pivot passwords and generates new passwords based on semantic similarity. Using the ABDPG sampling method, successful passwords are used as pivots for subsequent rounds, enhancing attack effectiveness. This explains FEPGuesser’s success on the more complex 000webhost dataset, positioning it well for modern password policies.

4) *Password Semantic Analysis*: FEPGuesser’s performance varies across datasets due to its reliance on semantic similarity. In this section, we analyze the semantic features of passwords in different datasets.

We use FID (Fréchet Inception Distance) [46], originally designed for GANs, to assess the quality of generated passwords. FID compares the distribution distances between generated and real data in feature space.

To analyze the semantic features of generated passwords, we utilize the encoded latent representations z from FEPGuesser. FID is particularly suitable for this purpose, as it measures the distance between Gaussian distributions—an approach that aligns well with the Gaussian assumption inherent in PassExBertVAE latent spaces. By calculating FID using the PassExBertVAE-encoded representations, we are able to assess semantic similarity more effectively than traditional metrics such as BLEU [47], which fail to capture nuances like synonymy (e.g., ‘football123’ vs. ‘soccer456’).

We compute the FID score between the training and testing sets using equation 4, as shown at the bottom of the page, where μ_{train} , μ_{test} represent the means, and C_{train} , C_{test} represent the covariances of the password latent space representations.

Table IX shows that 000webhost has a significantly higher FID score, indicating a more complex and diverse dataset, while Rockyou’s FID score is lower, suggesting less semantic difference. This matches our findings: FEPGuesser achieves a cracking rate of 36.34% on 000webhost and 63.15% on Rockyou.

Notably, RenRen’s FID score is higher than CSDN’s, indicating more semantic complexity, yet FEPGuesser achieves

higher cracking rates on RenRen. We speculate that RenRen’s higher frequency of non-tail passwords, as seen in Table VII, contributes to higher success rates by targeting high-frequency passwords.

V. DISCUSSION

In this section, we’ll delve into a discussion about this work and its potential future applications.

A. Semantic Information

The effectiveness of FEPGuesser is based on semantic similarity for attacks, explained as follows:

- 1) Passwords exhibit semantic similarity, as confirmed by previous studies [1], [48], showing that user passwords contain natural language patterns and similar groups share password components. In section IV-F4, we analyze password similarity across datasets using FID, linking semantic similarity to cracking success rates.
- 2) To capture both natural language and password-specific semantics, FEPGuesser fine-tunes a pre-trained language model with password data, effectively utilizing both features to capture implicit characteristics.

B. Pivot Passwords

FEPGuesser, as a semantic-based attack model, selects effective initial pivots—starting with training set passwords and expanding via ABDPG sampling. Its flexibility allows integration with PCFG by using PCFG-generated or cracked passwords as pivots, combining structural patterns with semantic insight. This creates a diverse and effective ensemble strategy, especially on datasets like Rockyou.

C. Credential Tweaking

Password tweaking attacks, which exploit users’ tendency to modify old passwords for new ones, have gained research interest recently [14]. Our PassExBertVAE can be applied in this context. Instead of encoding and perturbing a single password—risking the introduction of high-frequency patterns from the overall dataset—we propose encoding multiple of a user’s previous passwords. By interpolating in the latent space and decoding the result, we generate new passwords that better reflect user-specific behavior.

D. Complexity and Application Scenarios

Table X presents the runtime comparison of different deep learning models for generating 10^7 passwords. It should be noted that the PassFlow model exceeded 100 hours when generating 8×10^6 passwords, so we do not complete the full 10^7 password generation test. As shown in the results, our proposed FEPGuesser exhibits somewhat slower generation speed; however, Section IV-C has already demonstrated its superior cracking effectiveness.

$$d^2((\mu_{train}, C_{train}), (\mu_{test}, C_{test})) = \|\mu_{train} - \mu_{test}\|^2 + \text{Trace}\left(C_{train}C_{test} - 2(C_{train}C_{test})^{\frac{1}{2}}\right) \quad (4)$$

TABLE X
PASSWORD GENERATION RUNTIME ACROSS DEEP LEARNING MODELS

Model	Generation Runtime	Training Runtime	Generation Speed(pw/s)
FLA	3.18h	8.13h	873
PassGAN	650s	54.04h	15384.6
VAEPass	2h	2.2h	1388.9
PassFlow	100h+	3.83h	22.2
FEPGuesser	10h	3.98h	277.8

TABLE XI
PROPORTION OF SEMANTIC PASSWORDS IN THE DATASET

	CSDN	RenRen	000webhost	Rockyou
semantic rate (%)	34.05	20.41	72.88	75.92

We also provide the training time, and compared to other models, our proposed FEPGuesser has a shorter training duration. Additionally, we present the password generation speed under the scale of 10^7 passwords. Since some models can dynamically adjust the sampling range and the password generation speed is not constant, we calculate the average speed.

FEPGuesser excels in hybrid password guessing [10] by flexibly adapting to different password spaces through pivot password modification. It can use passwords guessed by other models as pivots to generate semantically related variations, naturally integrating features from other models. Additionally, FEPGuesser is suitable for white-hat security testing [49], efficiently reducing invalid attempts and conserving network resources when preparation time is sufficient.

VI. CONCLUSION

This work introduces FEPGuesser, a password guessing model that combines natural language processing with a VAE structure. It integrates a password pre-training layer with BERT using a gating mechanism to merge their representations. A VAE-based latent space controls semantic deviation, enabling the generation of new passwords through its decoder. Leveraging the observed “password semantic drift” phenomenon, FEPGuesser simulates real-world scenarios and improves attack efficiency using feedback information.

Attack experiments on multiple datasets show that FEPGuesser outperforms baseline models, particularly on the complex 000webhost dataset, which features longer passwords, more segments, and richer semantics. Cross-site attacks further demonstrate its adaptability to target datasets.

APPENDIX SEMANTIC PASSWORDS STUDY

This section analyzes the proportion of semantic passwords across four datasets. As shown in Table XI, we identify semantic passwords using WordNet [50], which includes English nouns, verbs, adjectives, and adverbs. A password is roughly classified as semantic if it contains any WordNet word.

It is worth noting that WordNet is designed for English vocabulary, making it more accurate for English datasets but

ineffective for Chinese datasets. Additionally, passwords may contain leetpeak (e.g., “p@ssw0rd”), which WordNet alone cannot fully account for in semantic analysis. Nevertheless, this method still provides valuable insights.

As shown in the results, the semantic password rate exceeds 70% in the English datasets (000webhost and Rockyou). For the Chinese datasets (CSDN and RenRen), the rates remain significant at 34.05% and 20.41%, respectively. Semantic passwords constitute a significant proportion of all password datasets.

REFERENCES

- [1] M. Weir, S. Aggarwal, B. De Medeiros, and B. Glodek, “Password cracking using probabilistic context-free grammars,” in *Proc. 30th IEEE Symp. Security Privacy*, Jan. 2009, pp. 391–405.
- [2] M. Dürmuth, F. Angelstorf, C. Castelluccia, D. Perito, and A. Chaabane, “OMEN: Faster password guessing using an ordered Markov enumerator,” in *Proc. Int. Symp. Eng. Secure Softw. Syst.* Cham, Switzerland: Springer, 2015, pp. 119–132.
- [3] B. Hitaj, P. Gasti, G. Ateniese, and F. Pérez-Cruz, “PassGAN: A deep learning approach for password guessing,” in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.* Cham, Switzerland: Springer, 2019, pp. 217–237.
- [4] D. Pasquini, A. Gangwal, G. Ateniese, M. Bernaschi, and M. Conti, “Improving password guessing via representation learning,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2021, pp. 1382–1399.
- [5] W. Yu et al., “A systematic review on password guessing tasks,” *Entropy*, vol. 25, no. 9, p. 1303, Sep. 2023.
- [6] W. D. Zou, “Advances on password guessing attack,” *J. Cryptologic Res.*, vol. 11, no. 1, pp. 67–100, 2024. [Online]. Available: <http://www.jcr.cacnet.org.cn/EN/10.13868/j.cnki.jcr.000670>
- [7] M. Weir. *PCFG*. [Online]. Available: https://github.com/lakiw/pcfg_cracker
- [8] M. Xu, C. Wang, J. Yu, J. Zhang, K. Zhang, and W. Han, “Chunk-level password guessing: Towards modeling refined password composition representations,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 5–20.
- [9] D. Wang, Z. Zhang, P. Wang, J. Yan, and X. Huang, “Targeted online password guessing: An underestimated threat,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1242–1254.
- [10] Z. Xie et al., “GuessFuse: Hybrid password guessing with multi-view,” *IEEE Trans. Inf. Forensics Security*, vol. 19, pp. 4215–4230, 2024.
- [11] T. Yang and D. Wang, “RankGuess: Password guessing using adversarial ranking,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2025, pp. 682–700.
- [12] M. Xu et al., “Improving real-world password guessing attacks via bi-directional transformers,” in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1001–1018.
- [13] Z. Xie, F. Shi, M. Zhang, Z. Rao, Y. Zhou, and X. Ji, “Similarities: The key factors influencing cross-site password guessing performance,” *Electronics*, vol. 14, no. 5, p. 945, Feb. 2025.
- [14] B. Pal, T. Daniel, R. Chatterjee, and T. Ristenpart, “Beyond credential stuffing: Password similarity models using neural networks,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 417–434.
- [15] W. Melicher et al., “Fast, lean, and accurate: Modeling password guessability using neural networks,” in *Proc. 25th USENIX Secur. Symp.*, 2016, pp. 175–191.
- [16] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of deep bidirectional transformers for language understanding,” 2018, *arXiv:1810.04805*.
- [17] W. Tai, H. T. Kung, X. Dong, M. Comiter, and C.-F. Kuo, “ExBERT: Extending pre-trained models with domain-specific vocabulary under constrained training resources,” in *Proc. Findings Assoc. Comput. Linguistics, EMNLP*, 2020, pp. 1433–1439.
- [18] R. Morris and K. Thompson, “Password security: A case history,” *Commun. ACM*, vol. 22, no. 11, pp. 594–597, Nov. 1979.
- [19] Openwall. (2013). *John the Ripper Password Cracker*. [Online]. Available: <https://www.openwall.com/john/>
- [20] Openwall. (2009). *Hashcat Advanced Password Recovery*. [Online]. Available: <https://hashcat.net/hashcat/>
- [21] R. Veras, C. Collins, and J. Thorpe, “On semantic patterns of passwords and their security impact,” in *Proc. NDSS*, 2014, pp. 1–16.

- [22] Z. Li, W. Han, and W. Xu, "A large-scale empirical analysis of Chinese web passwords," in *Proc. 23th USENIX Secur. Symp.*, 2014, pp. 559–574.
- [23] S. Houshmand, S. Aggarwal, and R. Flood, "Next gen PCFG password cracking," *IEEE Trans. Inf. Forensics Security*, vol. 10, no. 8, pp. 1776–1791, Aug. 2015.
- [24] C. Castelluccia, M. Dürmuth, and D. Perito, "Adaptive password-strength meters from Markov models," in *Proc. NDSS*, 2012, pp. 1–14.
- [25] A. Narayanan and V. Shmatikov, "Fast dictionary attacks on passwords using time-space tradeoff," in *Proc. 12th ACM Conf. Comput. Commun. Secur.*, Nov. 2005, pp. 364–372.
- [26] J. Xie, H. Cheng, R. Zhu, P. Wang, and K. Liang, "WordMarkov: A new password probability model of semantics," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, May 2022, pp. 3034–3038.
- [27] G. Pagnotta, D. Hitaj, F. De Gaspari, and L. V. Mancini, "PassFlow: Guessing passwords with generative flows," in *Proc. 52nd Annu. IEEE/IFIP Int. Conf. Dependable Syst. Netw. (DSN)*, Jun. 2022, pp. 251–262.
- [28] K. Yang, X. Hu, Q. Zhang, J. Wei, and W. Liu, "VAEPass: A lightweight passwords guessing model based on variational auto-encoder," *Comput. Secur.*, vol. 114, Mar. 2022, Art. no. 102587.
- [29] H. Li, M. Chen, S. Yan, C. Jia, and Z. Li, "Password guessing via neural language modeling," in *Proc. Int. Conf. Mach. Learn. Cyber Secur.* Cham, Switzerland: Springer, 2019, pp. 78–93.
- [30] Y. Liu et al., "RoBERTa: A robustly optimized BERT pretraining approach," 2019, *arXiv:1907.11692*.
- [31] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," 2019, *arXiv:1909.11942*.
- [32] M. Joshi, D. Chen, Y. Liu, D. S. Weld, L. Zettlemoyer, and O. Levy, "SpanBERT: Improving pre-training by representing and predicting spans," *Trans. Assoc. Comput. Linguistics*, vol. 8, pp. 64–77, Dec. 2020.
- [33] V. Lialin, V. Deshpande, X. Yao, and A. Rumshisky, "Scaling down to scale up: A guide to parameter-efficient fine-tuning," 2023, *arXiv:2303.15647*.
- [34] S.-A. Rebuffi, H. Bilen, and A. Vedaldi, "Learning multiple visual domains with residual adapters," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017, pp. 506–516.
- [35] N. Houlsby et al., "Parameter-efficient transfer learning for NLP," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 2790–2799.
- [36] J. Pfeiffer, A. Kamath, A. Rücklé, K. Cho, and I. Gurevych, "AdapterFusion: Non-destructive task composition for transfer learning," 2020, *arXiv:2005.00247*.
- [37] A. Bapna, N. Arivazhagan, and O. Firat, "Simple, scalable adaptation for neural machine translation," 2019, *arXiv:1909.08478*.
- [38] W. Li and J. Zeng, "Leet usage and its effect on password security," *IEEE Trans. Inf. Forensics Security*, vol. 16, pp. 2130–2143, 2021.
- [39] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," 2015, *arXiv:1508.07909*.
- [40] B. Li, H. Zhou, J. He, M. Wang, Y. Yang, and L. Li, "On the sentence embeddings from pre-trained language models," 2020, *arXiv:2011.05864*.
- [41] H. Fu, C. Li, X. Liu, J. Gao, A. Celikyilmaz, and L. Carin, "Cyclical annealing schedule: A simple approach to mitigating KL vanishing," 2019, *arXiv:1903.10145*.
- [42] M. Freitag and Y. Al-Onaizan, "Beam search strategies for neural machine translation," 2017, *arXiv:1702.01806*.
- [43] Y. Xiao and J. Zeng, "Dynamically generate password policy via Zipf distribution," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 835–848, 2022.
- [44] D. Wang, Y. Zou, Z. Zhang, and K. Xiu, "Password guessing using random forest," in *Proc. 32nd USENIX Secur. Symp. (USENIX Security)*, 2023, pp. 1–18.
- [45] GitHub.(2020). *Ignis-Sec/PWDB-Public: A Collection of Over 1 Billion Unique Cleartext Passwords*. [Online]. Available: <https://github.com/ignis-sec/Pwdb-Public>
- [46] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler, and S. Hochreiter, "GANs trained by a two time-scale update rule converge to a local Nash equilibrium," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 6629–6640.
- [47] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, Philadelphia, PA, USA, 2002, pp. 311–318. [Online]. Available: <https://aclanthology.org/P02-1040/>
- [48] A. S. Brown, E. Bracken, S. Zoccoli, and K. Douglas, "Generating and remembering passwords," *Appl. Cognit. Psychol.*, vol. 18, no. 6, pp. 641–651, Sep. 2004.
- [49] HackerOne.(2025). *White Hat Hackers: Techniques, Tools, and How to Become One*. [Online]. Available: <https://www.hackerone.com/knowledge-center/white-hat-hacker>
- [50] G. A. Miller, "WordNet: A lexical database for English," in *Proc. Workshop Hum. Lang. Technol.*, 1994, p. 1. [Online]. Available: <https://aclanthology.org/H94-1111/>



Qiuyan Qian received the B.S. degree (Hons.) from the School of Information, Xiamen University. She is currently pursuing the master's degree with Fudan University. Her research interests include password security, AI security, and natural language processing.



Jianping Zeng received the Ph.D. degree from Xiamen University in 2006. He is currently an Associate Professor with the School of Computer Science, Fudan University. He has published over 90 research papers in refereed journals and conferences in the areas of network security and artificial intelligence. He has authored four books. He holds ten patents. His research interests include big data security, AI security, and machine learning for social media analysis.